VoiceOps 2026: ZentroMart Voice Support Agent

Dharmana Purvi
LLM & Generative AI Course capstone, Consulting & Analytics Club, IIT Guwahati × Gnani.ai
GitHub repository: https://github.com/1purvi/VoiceOps

Overview

For this capstone I built a customer support agent for ZentroMart that answers questions from the company documents, handles a full conversation with order lookups and handovers, and talks to callers using Gnani's speech models.

Step	What I added	Result on the practice data
1	LLM with a system prompt only	12 of 52 questions correct
2	Search over the knowledge base (RAG)	52 of 52 questions correct
3	Tools, memory and caller verification	96 of 96 conversation turns correct
4	Gnani speech-to-text and text-to-speech	Working voice calls in English, Hindi and Hinglish

Step 1: Answering with an LLM

I used Nemotron (`nvidia/nemotron-3.5-super-vl-120b-a12b`) through the NVIDIA API with the openai Python client. The system prompt tells the model it is ZentroMart's AI support assistant, gives it today's date (20 October 2026) and asks for a JSON reply with the key facts, a decision (answer, clarify, escalate or refuse), the reply text and the sections it used. The first reply always tells the customer they are talking to an AI assistant.

Tech used: Python, openai client, NVIDIA API, pandas, Jupyter notebook.

Without any documents it got 12 of the 52 practice questions right. It did not make up policies, which was good, but for most questions it simply said it did not have the information and escalated (43 of the 46 questions that needed a real answer).

Step 2: Adding the company documents (RAG)

For chunking, I split each markdown file in `kb/` at its `[SECTION-ID]` headings, so one chunk is one policy section, and split it again at paragraph breaks if it was longer than 1,500 characters. Each chunk keeps its document type, status (current or superseded) and effective date. I also made each row of `products.csv` its own chunk so that questions about a product find the right item.

The chunks are embedded with multilingual-e5-base, which also handles Hindi, and stored as a numpy matrix saved with pickle, next to a BM25 keyword index. For each question I combine the two rankings with reciprocal rank fusion, move superseded and lower-priority documents slightly down, and send the top 26 sections to the LLM. The model is told to answer only from these sections and to say it does not know otherwise. Dates are calculated in Python and given to the model as a small table.

Tech used: sentence-transformers (intfloat/multilingual-e5-base), rank-bm25, numpy, pickle, pandas, Python datetime.

Metric	Step 1 (no documents)	Step 2 (RAG)
Fully correct	12 / 52 (23.1%)	52 / 52 (100%)
Decision accuracy	25.0%	100%
Fact recall	30.8%	100%
Policy violations	0	0
Retrieval recall@k / hit@k	-	0.933 / 1.000
MRR / citation precision	-	0.641 / 0.748

Every language improved (English from 9 to 33, Hindi from 2 to 8, Hinglish from 1 to 11), and the biggest gains were on refund timelines and return windows. My main takeaway from this comparison is that a good prompt makes the model careful, but only the documents make it useful.

Step 3: Holding a conversation

For conversations I turned the agent into a tool loop. The LLM can call `search_kb`, `lookup_order`, `create_ticket` and `date_calc` as many times as it needs, and then calls `final_answer`. It keeps the whole conversation in memory, including the order ID and whether the caller is verified, so a follow-up like "how long will the refund take?" works without the customer repeating anything.

Some rules I enforced in code instead of trusting the prompt: order details are shared only after the last 4 digits of the phone number or the pincode match (two attempts at most), the pincode itself is never sent to the LLM, there is one ticket per issue, and the agent only says it has handed over after the ticket is actually created.

Tech used: OpenAI-style function calling, the provided `tools.py`, the data CSV files, and the Step 2 search as a tool.

On the 23 practice conversations (96 turns) every turn was correct, including the tool calls (54 of 54 expected calls, no extra tickets). This held in English, Hindi and Hinglish and across all 12 scenarios, such as cancellations, refund status, compensation demands and asking for a human.

Step 4: Adding voice

`voice.py` wraps the same Step 3 agent. Gnani's prisma-v3.0 model does the speech-to-text and detects Hindi or English on its own; I also pass it shop words like refund and pincode, plus the previous sentence, to help it. Replies are spoken with Gnani timbre-v2.5 in the Kaveri voice, in the same language as the reply. Before speaking, the reply is adjusted for listening, for example order numbers are read in groups of four. If the audio is unclear, the agent asks the caller to repeat. Only Gnani is used for speech.

Tech used: gnani-vachana SDK, sounddevice, soundfile, numpy, Gradio.

Everything runs from one file. `python run.py` opens a web page where a call starts and you can ask questions by mic, and `python run.py --audio_dir <folder>` answers every audio file in a folder and saves the text reply, the audio reply and the document chunks it used.

In my own test call I started in English, verified my order, then switched to Hindi to ask about returning a pair of sneakers and then about the refund.

Problems I faced

Prompt tuning. Getting the prompt right took the most time. Early versions returned broken JSON or extra text around it, picked the wrong decision (escalating when they should have answered), or left out the exact number. What fixed it was a fixed JSON format, asking for the key facts before the reply, clear rules with examples for each decision, a parser that still finds the JSON when there is extra text, and retrying without JSON mode for providers that do not support it.

Choosing top k. I started with k = 6 and the results were very poor. Many questions need more than one section (a policy, a product row and an agent guide), and FAQ or older policy sections were taking up the slots. I ran an experiment increasing k from 6 towards

30 in steps of 5 (6, 11, 16, 21, 26) and adjusted the model temperature along the way. k = 26 gave the best results.

Conflicting documents. Some documents disagree. For example, the FAQ says all refunds take 5 to 7 days while the refund policy says wallet refunds are instant, and an older returns policy is still in the folder. Keeping the status and effective date on every chunk and giving the model a clear priority order (policy, agent guide, help, FAQ, marketing) solved this.

Dates. The model kept miscounting business days, so I moved all date calculations into Python.

Rate limits. Long prompts with 26 sections hit the token limits, so I added retries, pacing and a cache for every answer. The conversation agent searches 16 sections per query, and live calls search 8.

Speech. Speech-to-text sometimes got product names or short numbers wrong. The keyword and previous-sentence hints helped a lot.

What I would fix next

- The Step 3 retrieval scores look low (MRR 0.258, citation precision 0.038) even though the answers were right, mainly because the always-included agent guides are logged first. I would log only the search results.
- Voice replies can take 20 to 34 seconds on policy questions. I would use the faster mode everywhere, play a short filler while the agent thinks, and stream the audio.
- Some spoken replies are too long. I would keep them to two or three sentences.
- I tested Steps 1 and 2 on clean text. I would rerun them on the ASR text and keep some questions aside, to check I am not over-tuning to the practice set.

Other features

I also added a Gradio web app that starts a call as soon as it opens (`python run.py`), a hands-free terminal call that records on your voice, full Hindi and Hinglish support, an LLM provider that can be switched in `.env`, and cached evaluation runs so the results can be reproduced.